

10 — Résumé des 9 PDF + prompt Claude Code pour Phase C voie B / v5

Document de transition pour collaborer avec Claude Code sur la machine Linux. Il résume les neuf PDF déjà produits et fournit un prompt opérationnel pour demander à Claude Code d'améliorer la Phase C afin d'atteindre Illinois C pur, puis préparer T=10000.

But immédiat

Donner à Claude Code un contexte court, fiable et actionnable : état du projet, décisions prises, résultats validés, limites connues, et cahier des charges de développement pour la voie B / v5.

1. Résumé des 9 PDF

PDF	Titre / thème	Message essentiel
01	Cartographie diagnostic Riemann_Lab	Le projet possède plusieurs branches, des runs v3/v4, des scripts, du Wiki et des duplications à rationaliser. Il ne faut pas repartir de zéro mais conserver l'historique.
02	Plan migration / recovery / Brain Vault / RAG	Stratégie recommandée : conserver l'ancien, créer une structure propre hprzeta-lab, organiser recovery, RAG, Vault, données validées et scripts.
03	Checklist système Linux avant migration	Vérifier Linux, Git, GPU, Python, Ollama, partitions et sauvegardes avant tout nettoyage ou réinstallation.
04	Méthode scripts post-audit et matériel	Méthode d'extraction légère depuis l'archive d'audit, scripts assistant, mini-audit Git, remarques matériel zeta-lab.
05	Complément Git et priorités Phase 3	Mini-audit Git : branches confirmées, Riemann_Lab_C contient c_modules, priorité = sauvegarde, nettoyage, test Phase C.
06	Lexique IA & Brain Vault	Dictionnaire vivant : Vault, Brain Vault, RAG, Skills, agents, grounding, provenance, hallucination, artifacts.
07	Pavé primalité	Pilier 2 : nombres premiers, produit eulérien, $\pi(x)$, $\Lambda(n)$, $\psi(x)$, formule explicite, lien zéros $\leftrightarrow$ premiers.
08	CryptoZeta	Pilier 3 : cybersécurité défensive, RSA, ECC, post-quantique, hash/checksum, recovery, rôle indirect de ζ .
09	Synthèse journée Phase C v4 voie B	v4 hybride validée jusqu'à T=650 ; Illinois C pur non validé ; voie B demain pour réduire le biais Z_mpfr vs mpmath.

2. État technique actuel à transmettre à Claude Code

- Branche principale de développement Phase C : Riemann_Lab_C.
- c_modules compile et produit illinois_mpfr.so.
- test_illinois.py passe en mode hybride.
- benchmark_illinois.py montre un gain C isolé $\times 48.73$ sur $t \approx 500-638$.
- compute_zeros_v4.py fonctionne sur T=80, T=300 et T=650.
- Turing-Backlund : complet, aucun zéro manquant sur T=80/T=300/T=650.
- LMFDB : 19/20 premiers zéros sous $1e-10$, zéro #20 à environ $8.06e-10$.
- Répartition effective : 100 % Illinois_C $\rightarrow$ mpmath, 0 % Illinois_C pur.
- Cause mesurée : gamma_c a des résidus mpmath.siegelz entre environ $5.3e-4$ et $6.7e-2$, trop loin du seuil $1e-8$.

3. Décision stratégique

Ne pas lancer T=10000 maintenant. Le prochain travail est la voie B : améliorer la cohérence de Z_mpfr/Z_double avec mpmath.siegelz pour obtenir une proportion significative de zéros acceptés en Illinois C pur. Ensuite seulement : T=650 v5, T=1000 v5, puis T=10000.

4. Prompt prêt pour Claude Code

Le prompt complet est fourni dans le fichier Markdown séparé : prompt_claude_code_phase_c_voie_b_v5.md. Il est aussi reproduit ci-dessous pour archivage.

PROMPT CLAUDE CODE — Riemann_Lab Phase C / Voie B / v5

Tu es Claude Code exécuté sur la machine Linux `zeta-lab` dans le dépôt `~/projet\_zeta`, branche `Riemann\_Lab\_C`.

Mission principale
Améliorer la Phase C pour atteindre l'objectif : ****Illinois C pur fiable****, puis préparer une montée de tests jusqu'à ****T=10000****.

Contexte projet
Le projet Riemann_Lab/hprzeta calcule expérimentalement les zéros non triviaux de $\zeta(s)$. Il ne prétend jamais prouver l'hypothèse de Riemann. Il distingue toujours

État validé à ce jour
- `c\_modules` compile correctement : `make clean && make` produit `illinois\_mpfr.so`.

- `test\_illinois.py` passe sur les 10 premiers zéros en mode hybride.
- `benchmark\_illinois.py` mesure un gain isolé C/libmpfr d'environ $\times 48.73$ face à mpmath.findroot sur $t \approx 500-638$.
- `compute\_zeros\_v4.py` fonctionne sur $T=80$, $T=300$ et $T=650$.
- Turing-Backlund est complet sur les runs $T=80/T=300/T=650$: aucun zéro manquant.
- LMFDB reste à 19/20 sous $1e-10$, avec le zéro #20 à environ $8.06e-10$.
- Tous les runs v4 utilisent 100 % `Illinois\_C` → mpmath : `Illinois\_C` pur est à 0 %.

Cause technique identifiée

`compute\_zeros\_v4.py` accepte `gamma\_c` comme `Illinois\_C` pur seulement si :

```
```python
abs(float(mpmath.siegelz(gamma_c))) < 1e-8
```
```

Le diagnostic `phase\_c\_gamma\_c\_vs\_mpmath.txt` montre que `gamma\_c` produit par `illinois\_mpfr.so` est trop éloigné du vrai zéro `mpmath.siegelz` :

- moyenne $|\delta \gamma_c - \gamma_{\text{true}}| \approx 9.17e-03$;
- médiane $|\delta \gamma_c| \approx 8.35e-03$;
- max $|\delta \gamma_c| \approx 2.53e-02$;
- plus petit $|\text{abs}(\text{mpmath.siegelz}(\gamma_c))| \approx 5.30e-04$, très au-dessus du seuil $1e-8$.

Conclusion : ne pas assouplir le seuil. Il faut réduire le biais entre `Z\_mpfr` / `Z\_double` et `mpmath.siegelz`.

Fichiers importants

- `src/calculs/optimisation/compute\_zeros\_v4.py`
- `src/calculs/optimisation/c\_modules/illinois\_mpfr.c`
- `src/calculs/optimisation/c\_modules/z\_function.c`
- `src/calculs/optimisation/c\_modules/Makefile`
- `src/calculs/optimisation/c\_modules/test\_illinois.py`
- `src/calculs/optimisation/c\_modules/benchmark\_illinois.py`
- `src/calculs/optimisation/riemann\_siegel\_batch.py`
- `src/calculs/optimisation/theta\_rapide.py`
- `src/calculs/optimisation/turing\_validation.py`

Travail demandé — étape 1 : diagnostic comparatif

Créer un script de diagnostic non destructif :

`src/calculs/optimisation/c\_modules/diagnostic\_zmpfr\_vs\_mpmath.py`

Ce script doit produire un CSV avec colonnes :

```
```text
t, Z_double, Z_mpfr, mpmath_siegelz, abs_Zdouble_minus_mpmath, abs_Zmpfr_minus_mpmath, gamma_c, gamma_true, delta_gamma, residu_c
```
```

Échantillonner :

- autour des 20 premiers zéros ;
- autour de $t \approx 300$;
- autour de $t \approx 500-650$;
- éventuellement autour de $t \approx 1000$.

Objectif : caractériser précisément le biais de la fonction C.

Travail demandé — étape 2 : revue math/code

Auditer précisément :

- `theta\_double` et `theta\_mpfr` ;
- formule du terme de reste Riemann-Siegel $C_0 + C_1$;
- signe $(-1)^{(N-1)}$;
- convention $N = \text{floor}(\sqrt{t/(2\pi)})$;
- cohérence entre `Z\_double`, `Z\_mpfr`, `riemann\_siegel\_batch.py` et `mpmath.siegelz` ;
- correction Illinois : vérifier si la correction anti-stagnation doit être symétrique selon le côté stagnant.

Travail demandé — étape 3 : patch expérimental v4b/v5

Ne casse pas `compute\_zeros\_v4.py` validé. Créer une branche ou des fichiers expérimentaux :

- `compute\_zeros\_v5.py` ou `compute\_zeros\_v4b.py` ;
- éventuellement `z\_function\_v2.c` / `illinois\_mpfr\_v2.c` si nécessaire ;
- garder `v4` comme référence hybride validée.

Travail demandé — étape 4 : critères d'acceptation

Avant tout $T=10000$, il faut obtenir :

1. `make clean && make` OK ;
2. `test\_illinois.py` OK ;
3. `benchmark\_illinois.py` OK ;
4. `gamma\_c` avec $|\text{abs}(\text{mpmath.siegelz}(\gamma_c))| < 1e-8$ sur une proportion significative ;
5. run $T=650$ avec `Illinois\_C` pur $> 0\%$;
6. Turing complet ;
7. LMFDB stable ;
8. seulement ensuite $T=1000$, puis $T=10000$.

Contraintes absolues

- Ne pas prétendre prouver RH.
- Ne pas supprimer les runs validés.
- Ne pas casser v4 validée.
- Ne pas assouplir arbitrairement le seuil $1e-8$.
- Ne pas lancer $T=10000$ avant validation v5 sur $T=650$ et $T=1000$.
- Produire des logs reproductibles et des CSV de diagnostic.

Livrables attendus

- patch clair ou nouveaux fichiers v5 ;
- CSV diagnostic ;

- log `make/test/benchmark` ;
- log run T=650 v5 ;
- rapport court Markdown : `docs/phase\_c\_voie\_b\_v5\_plan.md`.

[... suite identique dans le fichier Markdown joint ...]

5. Fichiers à donner à Claude Code dans le dépôt

À ouvrir / analyser dans Claude Code :

```
~/projet_zeta/src/calculs/optimisation/compute_zeros_v4.py
~/projet_zeta/src/calculs/optimisation/c_modules/illinois_mpfr.c
~/projet_zeta/src/calculs/optimisation/c_modules/z_function.c
~/projet_zeta/src/calculs/optimisation/c_modules/Makefile
~/projet_zeta/src/calculs/optimisation/c_modules/test_illinois.py
~/projet_zeta/src/calculs/optimisation/c_modules/benchmark_illinois.py
~/projet_zeta/src/calculs/optimisation/riemann_siegel_batch.py
~/projet_zeta/src/calculs/optimisation/theta_rapide.py
~/projet_zeta/src/calculs/optimisation/turing_validation.py
```

Logs utiles :

```
~/phase_c_gamma_c_vs_mpmath.txt
runs v4_T80, v4_T300, v4_T650 dans calculs/
```

6. Livrables à demander à Claude Code

- Un script diagnostic_zmpfr_vs_mpmath.py produisant un CSV comparatif.
- Une revue courte des causes probables du biais Z_mpfr vs mpmath.siegelz.
- Une proposition de patch v5 sans casser v4.
- Une correction ou amélioration expérimentale de z_function.c / illinois_mpfr.c si justifiée.
- Un run de revalidation : make, test_illinois.py, benchmark_illinois.py.
- Un run T=650 v5 avec statistiques de répartition Illinois_C pur vs hybride.
- Un rapport docs/phase_c_voie_b_v5_plan.md.

7. Phrase de lancement recommandée

Claude Code, lis prompt_claude_code_phase_c_voie_b_v5.md.
Travaille uniquement sur la branche Riemann_Lab_C.
Ne casse pas compute_zeros_v4.py validé.
Crée un diagnostic du biais Z_mpfr vs mpmath.siegelz,
puis propose une voie v5 pour obtenir Illinois C pur avant tout test T=10000.

8. Conclusion

Ce document sert de passerelle entre les PDF de cadrage et le travail concret de codage dans Claude Code. Le cap est clair : conserver v4 hybride comme référence fiable, créer v5 expérimental, réduire le biais gamma_c, obtenir Illinois C pur, puis préparer T=10000.